Reviewer Pack — Coxeter-Dynkin Diagrams and Tree-Like Resolution Proof Size

Entry 1c63c0bfe316 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 07:41:23 UTC

Coxeter-Dynkin Diagrams and Tree-Like Resolution Proof Size

Entry ID: `1c63c0bfe316` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 07:41:23 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Algebra (Coxeter Groups) **Field B** (complexity object): Complexity Theory: Tree-like Resolution Proof Complexity

Statement:

> {'main': 'For any given instance of a satisfiability problem (SAT) represented as a tree-like resolution proof, the number of vertices in its associated Coxeter-Dynkin diagram is upper bounded by a polynomial function of the size of the proof.', 'invariant': 'The number of vertices in the Coxeter-Dynkin diagram associated with a tree-like resolution proof of a SAT instance is at most c(n)^2 * log(n), where n is the size of the proof and c(n) is a constant depending on n.', 'counterexample': 'There exists a counterexample if there is an instance of SAT such that its corresponding tree-like resolution proof has a Coxeter-Dynkin diagram with more than c(n)^2 * log(n) vertices for some constant c(n).'}

Rationale (proposer's reasoning):

> {'main': 'The relationship between Coxeter groups and combinatorial structures suggests that certain algebraic properties of the diagrams might reflect structural aspects of computational processes like resolution proofs.', 'bridge': 'Coxeter-Dynkin diagrams encode structural information about the relationships within a set, which could be related to the complexity of resolving logical statements in a proof tree.'}

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0e998503a1cb4b50`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The number of vertices in the associated Coxeter-Dynkin diagram is at most c(n)^2 * log(n), where n is the size of the proof and c(n) is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_random_sat_instance(n):
    variables = set()
    clauses = []
    for _ in range(n):
        clause = [random.choice([1, -1]) * (i + 1) for i in range(random.randint(2, n))]
        variables.update(abs(x) for x in clause)
        clauses.append(clause)
    return list(variables), clauses

def dpll_solve(instance, assignment):
    variables, clauses = instance
    free_variables = [v for v in variables if v not in assignment]
    if not free_variables:
        unsatisfiable = any(all(assignment.get(v, False) == (c > 0)) or all(assignment.get(v, Fals
        return not unsatisfiable
    v = free_variables[0]
    assignment[v] = True
    if dpll_solve(instance, assignment):
        return True
    del assignment[v]
    assignment[v] = False
    return dpll_solve(instance, assignment)

def construct_tree_like_resolution_proof(instance):
    variables, clauses = instance
    proof = []
    for v in variables:
        unit_clause = next((c for c in clauses if len(c) == 1 and abs(c[0]) == v), None)
        if unit_clause:
```

```
            proof.append(unit_clause)
            clauses.remove(unit_clause)
        else:
            clause = random.choice(clauses)
            proof.append(clause)
            clauses.remove(clause)
    return proof

def extract_coxeter_dynkin_diagram(proof):
    diagram = {}
    for clause in proof:
        for literal in clause:
            if abs(literal) not in diagram:
                diagram[abs(literal)] = set()
            for other_literal in clause:
                if abs(other_literal) != abs(literal):
                    diagram[abs(literal)].add(abs(other_literal))
    return diagram

def count_vertices_in_diagram(diagram):
    return sum(len(neighbors) for neighbors in diagram.values())

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        instances_tested = 0
        total_vertices = 0

        while instances_tested < 30:
            instance = generate_random_sat_instance(n)
            if dpll_solve(instance, {}):
                proof = construct_tree_like_resolution_proof(instance)
                diagram = extract_coxeter_dynkin_diagram(proof)
                vertices = count_vertices_in_diagram(diagram)
                total_vertices += vertices
                instances_tested += 1

        mean_vertices = total_vertices / instances_tested
        upper_bound = n**2 * math.log(n)

        results.append({
            "n": n,
            "mean_vertices": mean_vertices,
            "upper_bound": upper_bound,
            "conjecture_holds": mean_vertices <= upper_bound
        })

    metric_value = sum(res["mean_vertices"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    return {
        "metric_name": "Mean vertices in Coxeter-Dynkin diagram",
        "metric_value": metric_value,
```

```
        "instances_tested": 30 * len(n_values),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": "" if support_fraction >= 0.8 else f"Mean vertices {metric_value} > uppe
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=NA support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mean vertices > upper bound' first_failing_seed=
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b22ae58.py", line 113, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b22ae58.py", line 80, in run_trial
    if dpll_solve(instance, {}):
       ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b22ae58.py", line 35, in dpll_solve
    if dpll_solve(instance, assignment):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b22ae58.py", line 35, in dpll_solve
    if dpll_solve(instance, assignment):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b22ae58.py", line 35, in dpll_solve
    if dpll_solve(instance, assignment):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 2 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b22ae58.py", line 31, in dpll_solve
   unsatisfiable = any(all(assignment.get(v, False) == (c > 0)) or all(assignment.get(v, False) == (c
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b22ae58.py", line 31, in <genexpr>
   unsatisfiable = any(all(assignment.get(v, False) == (c > 0)) or all(assignment.get(v, False) == (c
                                            ^
NameError: cannot access free variable 'v' where it is not associated with a value in enclosing scope
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15590
2	propose	ollama_remote	glm4:latest	0	0	12142
3	preregistration	ollama_remote	glm4:latest	0	0	5814
4	novelty	ollama_remote	glm4:latest	0	0	5050
5	novelty	ollama_remote	glm4:latest	0	0	5160
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19118
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11159
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10097
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12525
10	judge	ollama_remote	glm4:latest	0	0	11089

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 107741 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/1c63c0bfe316.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1c63c0bfe316.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1c63c0bfe316.tar.gz` (if generated)